

1.数据

1.1字面量与变量

字面量类型及书写格式		
数字类型	整数(int)	10 18 -5 0
	浮点数/小数(float)	8.5 3.14 1.0 -3.5
布尔(bool)	表达现实生活中的逻辑, 真或假	True False
字符串(str)	描述文本的一种数据类型	"人生苦短, 我用Python"
空值(NoneType)	表示空或无值, 仅包含一个值None	None
数据容器	存储多项数据的容器	列表/元组/集合/字典 (后面详细讲解)

bool类型运算时为int; True为1,false为0;

变量的定义

- 变量: 程序中用来存储单个数据的容器, 通常会经常发生变化的数据存储在变量中。
- 定义格式: **变量名 = 变量的值**

num = 1114.1
num = 1114.2

num
1114.2

变量名: 每一个容器(空间)的名字
赋值: 表示将等号右侧的值, 赋予左侧的变量
变量值: 每一个变量都有自己存储的值(数据)

python是动态语言,会自动分析变量类型,定义变量时不用定义类型;

py的变量定义时必须赋值;

可以同时赋值多个变量,如: a,b=1,2;

1.2标识符

- 标识符 是程序员在代码中为变量、函数、类等元素所起的名字。 **变量名 = 变量的值**
- 命名规则(规定):
 - 只能包含字母(a-z, A-Z)、数字(0-9)、下划线(_)
 - 不能以数字开头
 - 不能使用关键字: True、False、None、and、or、if、else、elif、for、while等
 - 严格区分大小写, 比如: age, Age, AGE是三个变量
- 命名规范(变量):
 - 见名知意
 - 多个部分使用下划线连接
 - 英文字母全小写

name age color a a1 a2
update_time my_name user_first_name updatetime myname

蛇形命名法

1.3.数据类型:

标准名称	描述	说明
int	整数	数字类型, 存放整数, 如: 10 -5
float	浮点数	数字类型, 存放小数, 如: 8.5 3.14 1.0 -3.5
str	字符串	用引号引起来的都是字符串, 如: "Python"
bool	布尔	布尔类型, 描述真和假, 如: True False
NoneType	空值	表示空或无值, 仅包含一个值None

变量只是容器,查看的是存储的数据的类型;

1. 通过type()语句来得到数据的类型，具体语法为：type(要查看类型的数据)

```
# type(..)
print(type("Hello"))
print(type(10))
print(type(3.14))
```

```
# type(..)
num = 5.0
print(num)
print(type(num))
```

2. 通过isinstance()检查数据是否属于指定的类型，返回的是一个bool值，具体语法为：isinstance(数据, 类型)

```
# type(..)
num = 5.0
print(num)
print(isinstance(num, int))
```

1.3.1字符串

字符串

- 字符串的三种定义方式：

```
# 双引号定义
s1 = "Hello"

# 单引号定义
s2 = 'It\'s very interesting'

# 三引号定义(多行字符串)
s3 = """
尊敬的客户：
感谢您选择我们的产品。
我们讲会为您竭诚的服务。
祝好 ~
"""
```

- 常见的转义字符：

转义字符	名称	作用
\'	单引号	表示单引号 '
\"	双引号	表示双引号 "
\n	换行符	开始新的一行(换行)
\t	制表符	增加缩进，缩进一个制表符 (tab) 的大小

字符串拼接

- 很多时候，我们需要将多个字符串拼接起来，可以直接使用(+)进行拼接，方式如下：

```
slogan = "黑马程序员" + "成就IT黑马" # 多个字符串变量直接写
print(slogan)

slogan = "黑马程序员" + "成就IT黑马" # + 号拼接
print(slogan)
```

```
s1 = "人生苦短"
s2 = "我用Python"

print("吉多·范罗苏姆：" + s1 + "，" + s2)
```

非字符串类型用str()来进行类型转换后才可拼接；

字符串的格式化输出：

字符串格式化

- 通过 %占位符 的形式完成字符串和变量的快速拼接。（其中 % 表示我要占位，s表示将变量转为字符串放入占位的位置）

```
s1 = "涛哥"

print("大家好，我是%s，欢迎大家进入Python课程的学习" % s1)
```

```
s1 = "人生苦短"
s2 = "我用Python"

print("吉多·范罗苏姆：%s，%s" % (s1, s2))
```

- 也可以通过 f"内容{变量/表达式}" 的形式来完成快速格式化。

```
name = "涛哥"

print(f"大家好，我是 {name}，欢迎大家进入Python课程的学习")
```

```
s1 = "人生苦短"
s2 = "我用Python"

print(f"吉多·范罗苏姆：{s1}，{s2}")
```

推荐

1.4.输入与输出

输入

- input语句(函数)的功能就是获取键盘输入的数据,具体的用法为:`s = input(提示信息)`
- print语句(函数)的功能就是将数据输出到控制台,具体语法为:`print(数据..)`

```
# input(..)
name = input("请输入您的姓名: ")
print(f"欢迎您, {name}")

age = input("请输入您的年龄: ")
print(f"您今年 {age} 岁")
```

```
请输入您的姓名: 涛哥
欢迎您, 涛哥
请输入您的年龄: 18
您今年 18 岁
```

{num:.2f}可定义输出变量格式;

input到变量中的 数据都是 字符串类型;

- 其他类型转为int类型: `int(..)`
- 其他类型转为str类型: `str(..)`
- 其他类型转为float类型: `float(..)`
- 其他类型转为bool类型: `bool(..)`

类型转换:

1.5.运算符

1.5.1算数运算符:

算术运算符

- 算术运算符是用于执行基本的数学运算的符号,作用于一个或多个操作数上,并产生一个计算结果。

运算符	描述	举例说明
+	加法	$1 + 5$: 1加上5
-	减法	$8 - 3$: 8减3
*	乘法	$3 * 8$: 3乘以8
/	除法	$10 / 5$: 10除以5, 除法结果是小数
//	整除	$10 // 3$: 10整除3, 整除结果为整数
%	取余/求模	$10 \% 3$: 10模于3, 结果为1 (10除3取余数)
**	幂指数	$10 ** 3$: 10的3次方



可以通过`age=int(input())`来直接将输入转换类型;

float的运算是有可能出现 精度损失的(由于计算机底层无法表示所有的小数);

py中没有++/--的自运算符;

1.5.2赋值运算符:

赋值运算符

- 赋值运算符是编程语言中用于将值(或表达式的结果)保存到变量中的运算符。(把右边的值,赋给左边的变量)

运算符	描述	实例
=	赋值运算符	把 = 号右边的结果赋值给左边的变量, 如 <code>num = 1+2</code> , 结果num的值为3
+=	加法赋值运算符	<code>num += 2</code> 等效于 <code>num = num + 2</code>
-=	减法赋值运算符	<code>num -= 2</code> 等效于 <code>num = num - 2</code>
*=	乘法赋值运算符	<code>num *= 2</code> 等效于 <code>num = num * 2</code>
/=	除法赋值运算符	<code>num /= 2</code> 等效于 <code>num = num / 2</code>
%=	取模赋值运算符	<code>num %= 2</code> 等效于 <code>num = num % 2</code>
//=	取整除赋值运算符	<code>num //= 2</code> 等效于 <code>num = num // 2</code>
**=	幂赋值运算符	<code>num **= 2</code> 等效于 <code>num = num ** 2</code>

1.5.3比较运算符:

比较运算符

- 比较运算符，也称为关系运算符，用于比较两个值之间的关系。会计算运算符两边的表达式，然后返回一个布尔值作为结果(True - 表示比较关系成立; False - 表示比较关系不成立)。

运算符	描述	实例
==	等于	a == b 判断a是否等于b
!=	不等于	a != b 判断a是否不等于b
>	大于	a > b 判断a是否大于b
>=	大于等于	a >= b 判断a是否大于等于b
<	小于	a < b 判断a是否小于b
<=	小于等于	a <= b 判断a是否小于b

1.5.4逻辑运算符

逻辑运算符

- 逻辑运算符是用于连接多个条件(布尔)表达式(其值为“真”或“假”)，并返回一个最终布尔结果的运算符。

运算符	描述	实例
and	逻辑与(并且)	同时成立才是符合条件的(左右两边都为True, 结果才为True)
or	逻辑或(或者)	只要有一个符合条件的即可(只要左右两边有一个为True, 结果就为True)
not	逻辑非(取反)	取反操作, True变为False, False变为True

简化链式:

```
print(f"{n}在10-20之间:", 10 <= n <= 20) # and连接的条件是并且的关系, 两个条件同时成立(True), 结果才是True; 否则, 就是False
```

2. 控制语句

2.1 if语句

条件的返回 一定是bool类型

if条件判断

我们已经清楚了

如果我的高考分数超过680, 我就去清华读书。

- 场景: 只有满足指定条件, 才会执行对应的代码逻辑。

```
# if条件判断基本格式

if 要判断的条件:
    条件成立时, 要执行对应的操作
```

判断语句的结果, 必须是布尔类型True或False
True 会执行if内的代码
False则不会执行

```
# 定义变量
score = 695
# 条件判断
if score > 680:
    print("欢迎你, 来清华读书")
```

```
# 定义变量
score = 695
# 条件判断
if score > 680:
    print("欢迎你, 来清华读书")
    print("也恭喜你即将踏入精彩的大学生活")
```

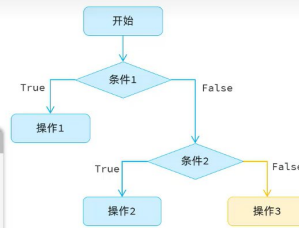
注意: Python中是通过缩进来描述代码的归属的, 归属于if代码块的语句, 需要在前方缩进4个空格(按Tab键Pycharm会自动转为空格)。

if语句进阶-if...elif...else

- 根据用户输入的数字，判断该数字是正数，还是负数，还是0？
 - 如果 数字 > 0 : 正数
 - 如果 数字 < 0 : 负数
 - 否则: 0

```
# if...elif...else结构
if 要判断的条件1:
    条件成立时，执行对应的操作1 ①
elif 要判断的条件2:
    条件成立时，执行对应的操作2 ②
else:
    条件不成立时，执行的操作3 ③

# if...elif...else结构
num = int(input("请输入数字: "))
if num > 0:
    print(f"{num} 是 正数")
elif num < 0:
    print(f"{num} 是 负数")
else:
    print(f"{num} 是 0")
```



注意: 满足第1个条件，就不会判断第2个部分和第3个部分；第1个条件不成立，才会判断第2个条件；1和2都不成立，才会进入else。

2. if...elif...else语句注意事项

- elif 可以写多个
- else 可有可无，如果有必须放在最后
- 多个条件判断是有顺序的，从上到下判断，如果前面有条件成立了，后面的条件不会判断了

:链式判断

空语句:pass

多分支:match case

模式匹配 match...case

```
# 工作日程安排
day = input("请输入星期几(1-7): ")
match day:
    case "1":
        print("周一: 工作会议日")
    case "2":
        print("周二: 学习培训日")
    case "3":
        print("周三: 项目开发日")
    case "4":
        print("周四: 代码审查日")
    case "5":
        print("周五: 总结规划日")
    case "6" | "7":
        print("周末: 休息放松")
    case _:
        print("输入错误")
```

执行流程:

1. 首先计算match指定的表达式的值。
2. 从上到下依次和case后面的值进行匹配，匹配正确，就执行对应语句。
3. 如果前面所有的case都没有匹配上，就会执行默认的案例 _。

也是链式判断

其中的 | 表示或的关系，匹配多个模式中的任意一个

匹配其他所有情况

2.2 while循环

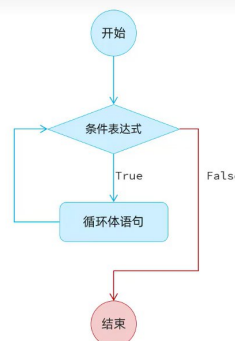
while

- while循环的语法结构:

```
# while语法结构
while 条件表达式:
    循环体语句1
    循环体语句2
    ...

# while语法结构
while 条件表达式:
    循环体语句1
    循环体语句2
    ...
else:
    条件为False, 循环正常结束时执行
```

- while循环的执行流程:



注意: while循环是通过条件表达式，来控制是否要进行下一次循环。

2.3 for循环

小结

1. for循环的语法格式

```
for 元素 in 待处理数据集:
    循环体代码(对元素进行处理)
else:
    不满足循环条件时, 执行的操作  可选
```

2. for循环与while循环的场景

- while循环: 用于在某个条件满足时一直循环, 循环的次数通常是未知的, 只知道循环开始/结束的条件。(关注的是循环的条件)
- for循环: 用于对一个已知的数据集进行遍历或已知次数的循环。(关注的是遍历每一个元素)

range语句

- 作用: 生成指定规则的数字序列
- 用法1: `range(end)` -> 获取一个从0开始, 到end结束的数字序列 (不含end本身)
 - `range(5)` 获取的数据就是 0,1,2,3,4
- 用法2: `range(start,end)` -> 获取一个从start开始, 到end结束的数字序列 (不含end本身)
 - `range(2,8)` 获取的数据就是 2,3,4,5,6,7
- 用法3: `range(start,end,step)` -> 获取一个从start开始, 到end结束的数字序列, step步长 (不含end本身)
 - `range(0,10,2)` 获取的数据就是 0,2,4,6,8

打印无换行: `print("★", end=" ")`

```
import random
```

获取随机数: `random_number = random.randint(a: 1, b: 100)`